DATABASE AUTOMATION USING VIEWS AND PROCEDURES

AIM:

To implement database automation in the Food ordering management Database System using Views, Stored Procedures, and Functions to simplify database operations and improve efficiency.

1. Customer Order Report View:

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_ORDER_REPORT AS SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM CUSTOMER C JOIN O
RDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID;

View created.
```

2. Product Sales Report View:

```
SQL> CREATE OR REPLACE VIEW PRODUCT_SALES_REPORT AS SELECT P.PRODUCT_NAME, NVL(SUM(OD.QUANTITY),0) AS QUANTITY_SOLD, NVL(SUM(OD.QUANTITY * OD.PRICE),0) AS T
OTAL_REVENUE FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME;

View created.
```

3. Payment Status View:

```
SQL> CREATE OR REPLACE VIEW PAYMENT_STATUS_VIEW AS SELECT ORDER_ID, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT FROM PAYMENT;

View created.
```

1. Add New Customer Order:

- Insert new order information.
- Add purchased product details.
- Calculate the total order amount.

```
SQL> CREATE OR REPLACE PROCEDURE ADD_CUSTOMER_ORDER(P_ORDER_ID IN VARCHAR2, P_CUSTOMER_ID IN VARCHAR2, P_PRODUCT_ID IN VARCHAR2, P_QUANTITY IN NUMBER, P_PAYMENT_ID IN VAR
CHAR2, P_PAYMENT_MODE IN VARCHAR2) AS V_PRICE NUMBER(10,2); V_TOTAL NUMBER(10,2); BEGIN SELECT PRICE INTO V_PRICE FROM PRODUCT WHERE PRODUCT_ID = P_PRODUCT_ID; V_TOTAL :=
 V_PRICE * P_QUANTITY; INSERT INTO ORDERS(ORDER_ID,CUSTOMER_ID,ORDER_DATE,TOTAL_AMOUNT,ORDER_STATUS) VALUES(P_ORDER_ID,P_CUSTOMER_ID,SYSDATE,V_TOTAL,'Pending'); INSERT IN
TO ORDER_DETAILS(ORDER_DETAIL_ID,ORDER_ID,PRODUCT_ID,QUANTITY,PRICE) VALUES('DET'||SUBSTR(P_ORDER_ID,4),P_ORDER_ID,P_PRODUCT_ID,P_QUANTITY,V_PRICE); INSERT INTO PAYMENT(P
AYMENT_ID,ORDER_ID,PAYMENT_DATE,PAYMENT_MODE,PAYMENT_STATUS,TRANSACTION_AMOUNT) VALUES(P_PAYMENT_ID,P_ORDER_ID,SYSDATE,P_PAYMENT_MODE,'Pending',V_TOTAL); COMMIT; END;
  2  /

Procedure created.
```

2. Update Order Status:

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_ORDER_STATUS(P_ORDER_ID IN VARCHAR2,P_NEW_STATUS IN VARCHAR2) AS BEGIN UPDATE ORDERS SET ORDER_STATUS=P_NEW_STATUS WHERE ORDER_ID=
P_ORDER_ID; IF SQL%ROWCOUNT=0 THEN RAISE_APPLICATION_ERROR(-20002,'Order ID Not Found'); END IF; COMMIT; END;
  2  /

Procedure created.
```

3. Generate Customer Purchase History:

```
SQL> CREATE OR REPLACE PROCEDURE GET_CUSTOMER_ORDERS(P_CUSTOMER_ID IN VARCHAR2,P_RESULT OUT SYS_REFCURSOR) AS BEGIN OPEN P_RESULT FOR SELECT C.CUSTOMER_NAME,O.ORDER_ID,O.
ORDER_DATE,O.TOTAL_AMOUNT,O.ORDER_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID WHERE C.CUSTOMER_ID=P_CUSTOMER_ID; END;
  2  /

Procedure created.
```

3.Create Functions for Sales Calculations:

A Function performs a specific calculation and returns a value.

The following functions are created:

1. Calculate Total Sales:

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_TOTAL_SALES(P_START_DATE IN DATE,P_END_DATE IN DATE) RETURN NUMBER AS V_TOTAL NUMBER; BEGIN SELECT NVL(SUM(TOTAL_AMOUNT),0) INTO
 V_TOTAL FROM ORDERS WHERE ORDER_DATE>=P_START_DATE AND ORDER_DATE<P_END_DATE+1; RETURN V_TOTAL; END;
  2  /

Function created.
```

2. Calculate Customer Total Spending:

```
SQL> CREATE OR REPLACE FUNCTION CUSTOMER_TOTAL_SPENDING(P_CUSTOMER_ID IN VARCHAR2) RETURN NUMBER AS V_SPENDING NUMBER; BEGIN SELECT NVL(SUM(TOTAL_AMOUNT),0) INTO V_SPENDI
NG FROM ORDERS WHERE CUSTOMER_ID=P_CUSTOMER_ID; RETURN V_SPENDING; END;
  2  /

Function created.
```

3. Calculate Product Revenue:

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_PRODUCT_REVENUE(P_PRODUCT_ID IN VARCHAR2) RETURN NUMBER AS V_REVENUE NUMBER; BEGIN SELECT NVL(SUM(QUANTITY*PRICE),0) INTO V_REVE
NUE FROM ORDER_DETAILS WHERE PRODUCT_ID=P_PRODUCT_ID; RETURN V_REVENUE; END;
  2  /

Function created.
```

4. Automate Repetitive Database Operations:

Database automation helps perform repeated operations automatically and reduces manual work.

1. Order Processing Automation:

When a customer places an order:

- Insert order details.
- Reduce product stock.
- Create a payment record.
- Update order status.

```
SQL> CREATE OR REPLACE TRIGGER CHECK_PRODUCT_STOCK BEFORE INSERT ON ORDER_DETAILS FOR EACH ROW DECLARE V_STOCK NUMBER; BEGIN SELECT STOCK_QUANTITY INTO V_STOCK FROM
CT WHERE PRODUCT_ID=:NEW.PRODUCT_ID; IF V_STOCK < :NEW.QUANTITY THEN RAISE_APPLICATION_ERROR(-20001,'Insufficient Product Stock'); END IF; END;
  2  /

Trigger created.
```

```
SQL> CREATE OR REPLACE TRIGGER REDUCE_PRODUCT_STOCK AFTER INSERT ON ORDER_DETAILS FOR EACH ROW BEGIN UPDATE PRODUCT SET STOCK_QUANTITY=STOCK_QUANTITY-:NEW.QUANTITY WHERE
PRODUCT_ID=:NEW.PRODUCT_ID; UPDATE INVENTORY SET STOCK_QUANTITY=STOCK_QUANTITY-:NEW.QUANTITY,STOCK_STATUS=CASE WHEN STOCK_QUANTITY-:NEW.QUANTITY<=5 THEN 'Low Stock' ELSE
'Available' END,LAST_UPDATED=SYSDATE WHERE PRODUCT_ID=:NEW.PRODUCT_ID; END;
  2  /

Trigger created.
```

2. Inventory Update Automation:

The inventory operations include:

- Increase stock after receiving new supply.
- Reduce stock after a purchase.
- Identify products with low stock.

```
SQL> CREATE OR REPLACE PROCEDURE ADD_NEW_STOCK(P_PRODUCT_ID IN VARCHAR2,P_QUANTITY IN NUMBER) AS BEGIN IF P_QUANTITY<=0 THEN RAISE_APPLICATION_ERROR(-20003,'Quantity Must
 Be Greater Than Zero'); END IF; UPDATE PRODUCT SET STOCK_QUANTITY=STOCK_QUANTITY+P_QUANTITY WHERE PRODUCT_ID=P_PRODUCT_ID; IF SQL%ROWCOUNT=0 THEN RAISE_APPLICATION_ERROR
(-20004,'Product ID Not Found'); END IF; UPDATE INVENTORY SET STOCK_QUANTITY=STOCK_QUANTITY+P_QUANTITY,STOCK_STATUS=CASE WHEN STOCK_QUANTITY+P_QUANTITY<=5 THEN 'Low Stock
' ELSE 'Available' END,LAST_UPDATED=SYSDATE WHERE PRODUCT_ID=P_PRODUCT_ID; COMMIT; END;
  2  /

Procedure created.
```

```
SQL> CREATE OR REPLACE VIEW LOW_STOCK_PRODUCTS AS SELECT PRODUCT_ID,PRODUCT_NAME,STOCK_QUANTITY FROM PRODUCT WHERE STOCK_QUANTITY<=5;

View created.
```

3. Report Generation Automation:

The system generates the following reports:

- Daily sales report.

```
SQL> CREATE OR REPLACE VIEW DAILY_SALES_REPORT AS SELECT TRUNC(ORDER_DATE) AS SALES_DATE,COUNT(ORDER_ID) AS TOTAL_ORDERS,NVL(SUM(TOTAL_AMOUNT),0) AS TOTAL_REVENUE FROM OR
DERS GROUP BY TRUNC(ORDER_DATE);

View created.
```

- Monthly revenue report.

```
SQL> CREATE OR REPLACE VIEW MONTHLY_REVENUE_REPORT AS SELECT TO_CHAR(ORDER_DATE,'YYYY-MM') AS SALES_MONTH,COUNT(ORDER_ID) AS TOTAL_ORDERS,NVL(SUM(TOTAL_AMOUNT),0) AS TOTA
L_REVENUE FROM ORDERS GROUP BY TO_CHAR(ORDER_DATE,'YYYY-MM');

View created.
```

- Customer purchase report.

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_PURCHASE_REPORT AS SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME,COUNT(O.ORDER_ID) AS TOTAL_ORDERS,NVL(SUM(O.TOTAL_AMOUNT),0) AS TOTAL_SPENDIN
G FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME;

View created.
```

5. Test Database Objects:

The created database objects are tested to verify their correctness.

Views Testing:

- Check correct data retrieval.
- Verify report accuracy.

```
SQL> SELECT * FROM CUSTOMER_ORDER_REPORT;

CUSTOMER_NAME             ORDER_ID     ORDER_DATE      TOTAL_AMOUNT ORDER_STATUS
------------------------- ------------ --------------- ------------ --------------------
Sanjay Kumar              ORD001       01-SEP-26             520.00 Delivered
Harini Devi               ORD002       02-SEP-26             250.00 Delivered
Praveen Raj               ORD003       03-SEP-26             600.00 Shipped
Keerthana S               ORD004       04-SEP-26             400.00 Delivered
Pavithra M                ORD006       06-SEP-26             850.00 Shipped
Ashwin Anand              ORD007       07-SEP-26             420.00 Delivered
Lavanya M                 ORD010       10-SEP-26             520.00 Shipped
Surya Prakash             ORD009       09-SEP-26             200.00 Pending
Nithya R                  ORD008       08-SEP-26             360.00 Pending
Dinesh Kumar              ORD005       05-SEP-26             180.00 Pending

10 rows selected.
```

```
SQL> SELECT * FROM PRODUCT_SALES_REPORT;

PRODUCT_NAME              QUANTITY_SOLD TOTAL_REVENUE
------------------------- ------------- -------------
Cotton Tote Bag                       3        540.00
Bamboo Tray                           1        400.00
Aloe Face Cream                       1        250.00
Steel Lunch Box                       1        600.00
Eco Cotton Kurta                      1        850.00
Organic Oats                          1        200.00
Natural Honey                         2        640.00
Eco Diary                             3        420.00
Bamboo Comb                           0           .00
Organic Seed Pack                     0           .00

10 rows selected.
```

```
SQL> SELECT * FROM PAYMENT_STATUS_VIEW;

ORDER_ID     PAYMENT_MODE         PAYMENT_STATUS  TRANSACTION_AMOUNT
------------ -------------------- --------------- ------------------
ORD001       UPI                  Success                        520
ORD002       Card                 Success                        250
ORD003       Net Banking          Success                        600
ORD004       UPI                  Success                        400
ORD005       Cash                 Pending                        180
ORD006       Card                 Success                        850
ORD007       UPI                  Success                        420
ORD008       Card                 Pending                        360
ORD009       Net Banking          Failed                         200
ORD010       UPI                  Success                        320

10 rows selected.
```

```
SQL> SELECT * FROM LOW_STOCK_PRODUCTS;

no rows selected
```

```
SQL> SELECT * FROM DAILY_SALES_REPORT;

SALES_DAT TOTAL_ORDERS TOTAL_REVENUE
--------- ------------ -------------
23-SEP-26            1           550
10-OCT-26            1           900
15-MAY-26            1          1250
10-MAY-26            1           890
09-MAR-26            1           480
```

Procedures Testing:

- Test procedure execution.
- Verify input parameters.
- Check output results.

```
SQL> EXEC ADD_CUSTOMER_ORDER('ORD011','CUS001','PRO001',2,'PAY011','UPI')

PL/SQL procedure successfully completed.
```

```
SQL> EXEC UPDATE_ORDER_STATUS('ORD011','Shipped');

PL/SQL procedure successfully completed.
```

```
SQL> SELECT * FROM MONTHLY_REVENUE_REPORT;

SALES_M TOTAL_ORDERS TOTAL_REVENUE
------- ------------ -------------
2026-09           10       4300.00
```

```
SQL> SELECT * FROM CUSTOMER_PURCHASE_REPORT;
CUSTOMER_ID
------------
CUSTOMER_NAME
--------------------------------------------------
----
TOTAL_ORDERS TOTAL_SPENDING
------------ ---------------
         105
KAMESH
           1             900

         102
VIJAY KUMAR
           1            1250
CUSTOMER_ID
------------
CUSTOMER_NAME
--------------------------------------------------
----
TOTAL_ORDERS TOTAL_SPENDING
------------ ---------------

         104
SURESH
           1             890

         103
ANITHA
CUSTOMER_ID
------------
CUSTOMER_NAME
--------------------------------------------------
----
TOTAL_ORDERS TOTAL_SPENDING
------------ ---------------
           1             480

         101
SNEHA
           1             550
```

```
SQL> EXEC GET_CUSTOMER_ORDERS('101',:RC);

PL/SQL procedure successfully completed.

SQL> PRINT RC;

CUSTOMER_NAME
--------------------------------------------------
----
ORDER_ID   ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ -------------
SNEHA
106        23-SEP-26          550 Pending
```

Functions Testing:

- Verify calculation accuracy.
- Check returned values

```
SQL> EXEC ADD_NEW_STOCK('PRO001',10);

PL/SQL procedure successfully completed.
```

```
SQL> SELECT CALCULATE_TOTAL_SALES(DATE '2026-01-01',DATE '2026-12-31') AS TOTAL_SALES FROM DUAL;

TOTAL_SALES
-----------
       4900
```

```
SQL> SELECT CALCULATE_PRODUCT_REVENUE('PRO001') AS PRODUCT_REVENUE FROM DUAL;

PRODUCT_REVENUE
---------------
            800
```

```
SQL> SELECT CUSTOMER_TOTAL_SPENDING('CUS001') AS TOTAL_SPENDING FROM DUA

TOTAL_SPENDING
--------------
          1120
```